# Apache Parquet

Co jest w środku pliku i dlaczego analityka działa dzięki niemu szybciej

Kamil Krawiec

# Czym jest Parquet

Apache Parquet to otwarty, kolumnowy format przechowywania danych, zaprojektowany z myślą o wydajnym przetwarzaniu i analizie dużych zbiorów danych. Wykorzystuje zaawansowane mechanizmy kodowania i kompresji, aby ograniczyć rozmiar plików i przyspieszyć odczyt danych



# Tabela zamówień

| id | data | klient | miasto | woj | kategoria | produkt | ilosc | cena | rabat | kwota | platnosc | status | kurier |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 1 | 14.01 | K102 | Kraków | MP | sport | 501 | 1 | 120 | 0 | 120 | BLIK | nowe | DPD |
| 2 | 14.01 | K311 | Kraków | MP | dom | 502 | 2 | 40 | 0 | 80 | BLIK | nowe | DPD |
| 3 | 15.01 | K102 | Kraków | MP | sport | 503 | 1 | 200 | 0 | 200 | karta | nowe | InPost |
| 4 | 15.01 | K451 | Tarnów | MP | moda | 504 | 1 | 50 | 0 | 50 | BLIK | nowe | InPost |
| 5 | 16.01 | K451 | Tarnów | MP | dom | 505 | 1 | 100 | 10 | 90 | karta | wysłane | DPD |
| 6 | 16.01 | K230 | Kraków | MP | sport | 506 | 3 | 50 | 0 | 150 | BLIK | wysłane | DPD |
| 7 | 17.01 | K887 | Warszawa | MZ | moda | 507 | 2 | 600 | 0 | 1200 | karta | wysłane | DHL |
| 8 | 17.01 | K887 | Warszawa | MZ | dom | 508 | 1 | 70 | 0 | 70 | BLIK | wysłane | DHL |

14 kolumn i 8 wierszy, czyli **112 wartości**.

# Układ wierszowy

```
SELECT SUM(kwota) FROM zamowienia
```

| | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 1 | 14.01 | K102 | Kraków | MP | sport | 501 | 1 | 120 | 0 | 120 | BLIK | nowe | DPD |
| 2 | 14.01 | K311 | Kraków | MP | dom | 502 | 2 | 40 | 0 | 80 | BLIK | nowe | DPD |
| 3 | 15.01 | K102 | Kraków | MP | sport | 503 | 1 | 200 | 0 | 200 | karta | nowe | InPost |
| 4 | 15.01 | K451 | Tarnów | MP | moda | 504 | 1 | 50 | 0 | 50 | BLIK | nowe | InPost |
| 5 | 16.01 | K451 | Tarnów | MP | dom | 505 | 1 | 100 | 10 | 90 | karta | wysłane | DPD |
| 6 | 16.01 | K230 | Kraków | MP | sport | 506 | 3 | 50 | 0 | 150 | BLIK | wysłane | DPD |
| 7 | 17.01 | K887 | Warszawa | MZ | moda | 507 | 2 | 600 | 0 | 1200 | karta | wysłane | DHL |
| 8 | 17.01 | K887 | Warszawa | MZ | dom | 508 | 1 | 70 | 0 | 70 | BLIK | wysłane | DHL |

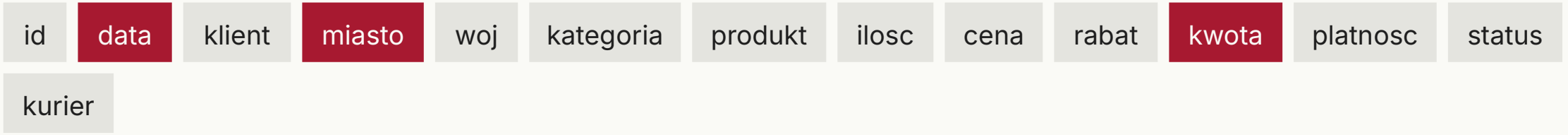
# Silnik przechodzi przez 112 z 112 wartości

# Układ kolumnowy: czytamy 8 z 112

| id | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 |
|---|---|---|---|---|---|---|---|---|
| data | 14.01 | 14.01 | 15.01 | 15.01 | 16.01 | 16.01 | 17.01 | 17.01 |
| klient | K102 | K311 | K102 | K451 | K451 | K230 | K887 | K887 |
| miasto | Kraków | Kraków | Kraków | Tarnów | Tarnów | Kraków | Warszawa | Warszawa |
| woj | MP | MP | MP | MP | MP | MP | MZ | MZ |
| kategoria | sport | dom | sport | moda | dom | sport | moda | dom |
| produkt | 501 | 502 | 503 | 504 | 505 | 506 | 507 | 508 |
| ilosc | 1 | 2 | 1 | 1 | 1 | 3 | 2 | 1 |
| cena | 120 | 40 | 200 | 50 | 100 | 50 | 600 | 70 |
| rabat | 0 | 0 | 0 | 0 | 10 | 0 | 0 | 0 |
| kwota | 120 | 80 | 200 | 50 | 90 | 150 | 1200 | 70 |
| platnosc | BLIK | BLIK | karta | BLIK | karta | BLIK | karta | BLIK |
| status | nowe | nowe | nowe | nowe | wysłane | wysłane | wysłane | wysłane |
| kurier | DPD | DPD | InPost | InPost | DPD | DPD | DHL | DHL |

# Jaki problem to rozwiązuje

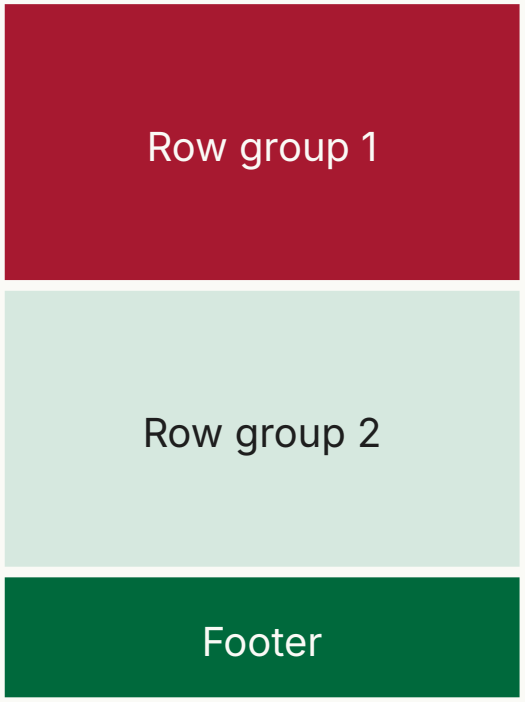
Analityka potrzebuje kilku kolumn z ogromnej tabeli, więc nie powinna czytać wszystkich.




```
SELECT miasto, SUM(kwota) ... WHERE data = ...
```

# Budowa pliku

## Plik




Footer: metadane na końcu pliku

## Row group




Grupa wierszy zapisana kolumnami

## Column chunk




Jedna kolumna w jednej grupie

## Page




Strona: najmniejsza kodowana porcja

# Nasz plik, bajt po bajcie



| Bajty | Co tam jest |
|---|---|
| 0–3 | znacznik PAR1 |
| 4–1283 | Row group 1: wiersze 1–4 |
| 1284–2606 | Row group 2: wiersze 5–8 |
| 2607–6558 | Footer: 3952 bajty metadanych |
| 6559–6562 | długość footera |
| 6563–6566 | znacznik PAR1 |

# Co jest w row group

## Row group 1: 4 wiersze, 1280 B, kwota od 50 do 200

| id | data | klient | miasto | woj | kategoria | produkt | ilosc | cena | rabat | kwota | platnosc | status | kurier |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 1 | 14.01 | K102 | Kraków | MP | sport | 501 | 1 | 120 | 0 | 120 | BLIK | nowe | DPD |
| 2 | 14.01 | K311 | Kraków | MP | dom | 502 | 2 | 40 | 0 | 80 | BLIK | nowe | DPD |
| 3 | 15.01 | K102 | Kraków | MP | sport | 503 | 1 | 200 | 0 | 200 | karta | nowe | InPost |
| 4 | 15.01 | K451 | Tarnów | MP | moda | 504 | 1 | 50 | 0 | 50 | BLIK | nowe | InPost |

## Row group 2: 4 wiersze, 1323 B, kwota od 70 do 1200

| | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 5 | 16.01 | K451 | Tarnów | MP | dom | 505 | 1 | 100 | 10 | 90 | karta | wysłane | DPD |
| 6 | 16.01 | K230 | Kraków | MP | sport | 506 | 3 | 50 | 0 | 150 | BLIK | wysłane | DPD |
| 7 | 17.01 | K887 | Warszawa | MZ | moda | 507 | 2 | 600 | 0 | 1200 | karta | wysłane | DHL |
| 8 | 17.01 | K887 | Warszawa | MZ | dom | 508 | 1 | 70 | 0 | 70 | BLIK | wysłane | DHL |

# Co jest w column chunk

**miasto, row group 1**

| |
|---|
| Kraków |
| Kraków |
| Kraków |
| Tarnów |

| Metadane | Wartość |
|---|---|
| typ fizyczny | BYTE_ARRAY |
| początek | bajt 284 |
| rozmiar | 86 B |
| min | Kraków |
| max | Tarnów |
| NULL | 0 |
| strony | słownik + dane |

# Co jest w page: prawdziwe bajty

## Strona słownika: 22 B

```
07 00 00 00 4b 72 61 6b c3 b3 77 07 00 00 00 54 61 72 6e c3 b3 77
```

długość 7, **Kraków** (numer 0), długość 7, **Tarnów** (numer 1)

## Strona danych: 9 B

```
02 00 00 00 08 01 01 03 08
```

**Pierwsze 6 bajtów**
4 razy „wartość jest”, czyli brak NULL

**01**
szerokość numeru: 1 bit

**03 08**
bity 0 0 0 1, czyli Kraków ×3, Tarnów

# Co jest w footerze

```
created_by: parquet-cpp-arrow version 26.0.1
num_columns: 14
num_rows: 8
num_row_groups: 2
format_version: 2.6
serialized_size: 3952
```

Do tego schemat kolumn, położenie każdego fragmentu i statystyki **min**, **max** oraz liczba **NULL**.

# Column pruning: 1,83 z 18,86 MB

| Kolumna | Rozmiar po kompresji Snappy |
|---|---|
| id | 6,03 MB |
| klient | 3,80 MB |
| produkt | 1,83 MB |
| kwota | 1,83 MB: tylko to czytamy |
| cena | 1,30 MB |
| data | 1,15 MB |
| ilosc | 0,51 MB |
| miasto, woj, kategoria, rabat, status | po 0,38 MB |
| platnosc, kurier | po 0,26 MB |

# Predicate pushdown

Warunek WHERE jest używany już przy odczycie, żeby pominąć niepotrzebne dane.

```
WHERE kwota > 1000
```

**Row group 1**
max kwoty: 200
**pomijamy**

**Row group 2**
max kwoty: 1200
**czytamy**

# Filtr na milionie wierszy

```
SELECT COUNT(*) FROM 'zamowienia.parquet' WHERE kwota > 4900
```

**Bez sortowania: statystyki nie wykluczają żadnej z 10 grup**




**Po sortowaniu według kwoty: zostaje 1 grupa z 10**



# Kodowanie słownikowe

| miasto |
| --- |
| Kraków |
| Kraków |
| Kraków |
| Tarnów |



| słownik |
| --- |
| 0 = Kraków |
| 1 = Tarnów |

| dane |
| --- |
| 0 |
| 0 |
| 0 |
| 1 |

Kodowanie to zapis wartości w sprytniejszej postaci, z wykorzystaniem wiedzy o danych.

# RLE i bit-packing

## RLE: zapis serii

`1 1 1 1`



**`08 01`**

seria 4 razy, wartość 1
(08 = 4 przesunięte o bit)

## Bit-packing: mało bitów

`0 0 0 1`



**`03 08`**

nagłówek i 1 bit na wartość
(08 = 00001000)

# Jak działa kompresja

Kodowanie zna dane. **Kompresja widzi tylko bajty**: szuka powtórzeń i zastępuje je odwołaniem wstecz.



# Snappy, gzip i ZSTD

| Algorytm | Co robi |
| --- | --- |
| Snappy | tylko odwołania wstecz |
| gzip | odwołania + kody Huffmana |
| ZSTD | odwołania + Huffman i FSE |

Kod Huffmana: częstsze symbole dostają krótszy zapis. FSE działa w podobnym celu, inną metodą.

# Ile to daje: rozmiar pliku

Bez słownika, bez kompresji: **118,4 MB**



Słownik, bez kompresji: **24,9 MB**



Słownik + Snappy: **18,9 MB**



Słownik + ZSTD: **14,4 MB**



# Ograniczenia

## Wolniejszy zapis

bufor, kodowanie, statystyki

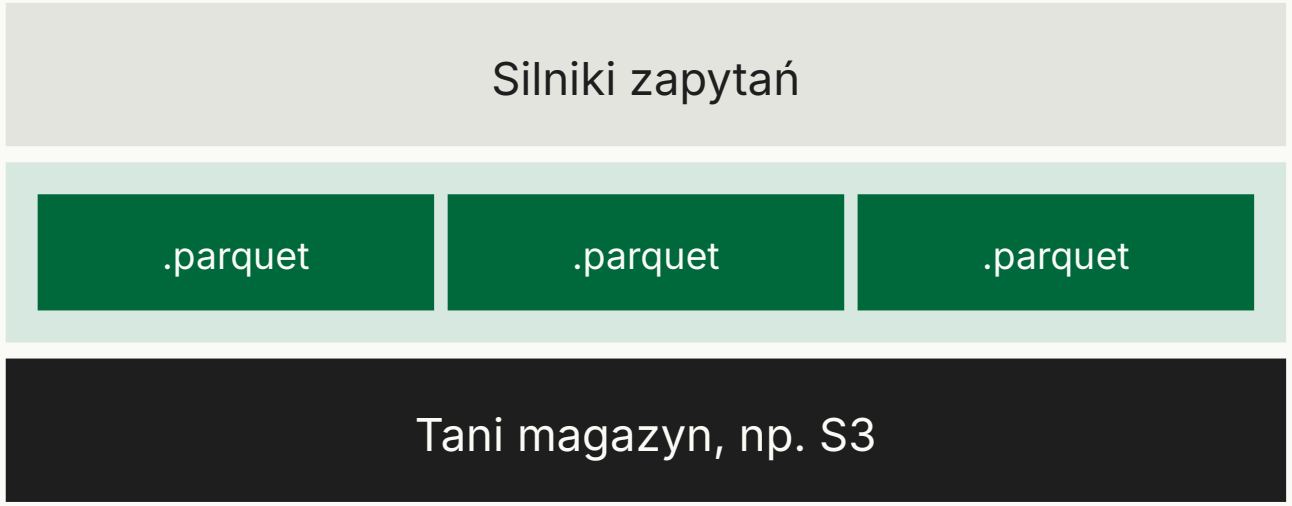
## Trudne aktualizacje

wartości zakodowane w stronach

## Małe pliki

narzut na każdy plik

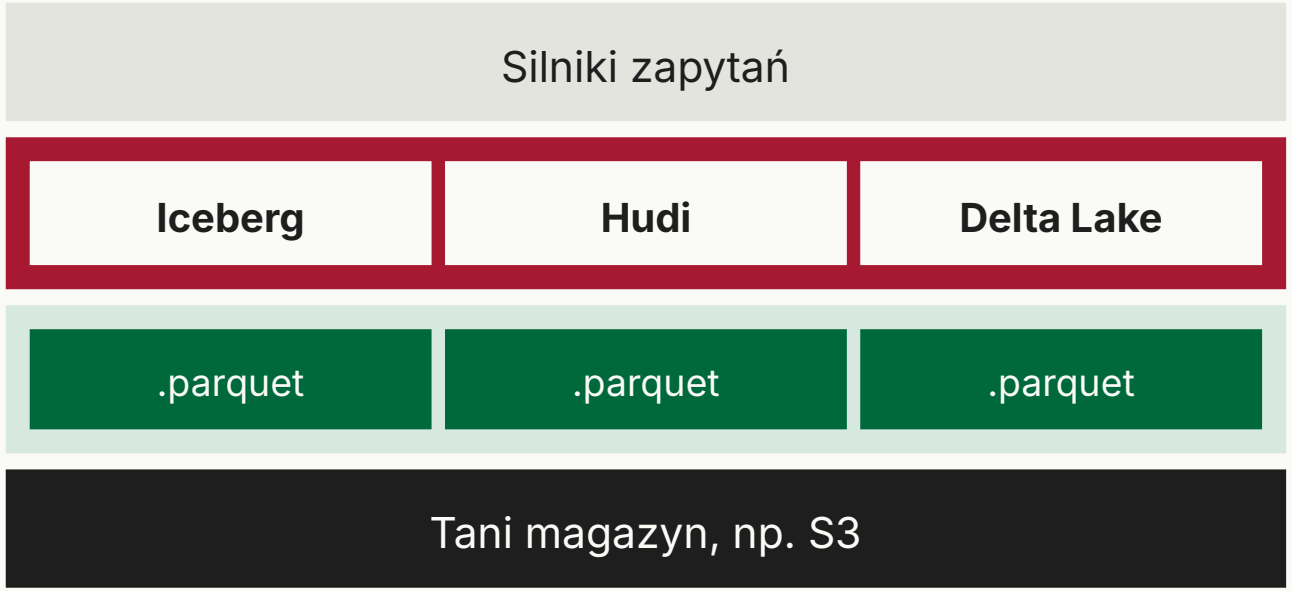
# Data lake na początku: same pliki




## Czego brakowało

- transakcji ACID
- historii zmian
- wygodnych UPDATE

# Potem doszła warstwa metadanych




- transakcje ACID
- podróż w czasie
- ewolucja schematu

**Dane nadal leżą w plikach.**